

WARNING

These slides are **not optimized for printing** or exam preparation. These are for lecture delivery only.

These slides contain a lot of animations. For optimal results, **watch in slideshow mode**.

Which one doesn't have an abstraction problem? Tip: check if the comment, name, parameters, return value **all fit together well**.

(a)

```
/**
 * UI uses this for formatting t
 */
String format(String s) {
```

Subjective. Context-dependent.
Focus on the reasoning
rather than the answer.

(b)

```
void execute(String delimiter) {
```

(c)

```
/**
 * @return list of persons.
 */
List<String[]> execute(Command command) {
```

(d)

```
/**
 * Executes the given command.
 * @return the Result of the execution.
 */
Result execute(Command command) {
```

Answer this in the Canvas quiz

Q₁

```
/**  
 *   UI uses this for formatting text.  
 */  
String format(String s) {
```

Good?

- ☐ Yes
- ☐ No

The comment refers to the user of the method

Q₂

```
void execute(String delimiter) {
```

Good?

- ☐ Yes
- ☐ No

Parameter doesn't fit.

Q₃

```
/**  
 * @return list of persons.  
 */  
List<String[]> execute(Command command) {
```

Good?

- ☐ Yes
- ☐ No

Return value doesn't fit.

Q₄

```
/**  
 * Executes the given command.  
 * @return the Result of the execution.  
 */  
Result execute(Command command) {
```

Good?

- ☐ Yes
- ☐ No

Everything fits; the method makes sense on its own.

All four might work as expected.
And yet, some are better abstractions than others ...
e.g., easily understood without needing additional info

might not matter if,	might make a big difference if,
500LoC	50,000 LoC
to the author	other developers
1 week later	1 year later

Picking the right abstractions
(for the problem at hand) is key.
If you can do that, you are half way there!

Oh No!!! **Where's my laptop?**



Mom: **What did
you do today?**

You:

I went to school,
then went to the canteen,
then I did an **unzipOutClose**
and ...





Coding is about communicating
[] with humans
[] with machines

Mom: What did
you do today?

You: **unzipOutClose**

unzipOutClose

unzipBagAndTakeOutThingAndCloseBag()

unzipBag()
takeOutThing()
closeBag()

When defining methods (classes, variables),
you are **defining a new vocabulary**, for readers

Using Java: **hard?**
Defining a new
language **well: harder!**

“ Every system is built from a domain-specific language designed by the programmers to describe that system. Functions are the verbs of that language, and classes are the nouns. — Robert C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*

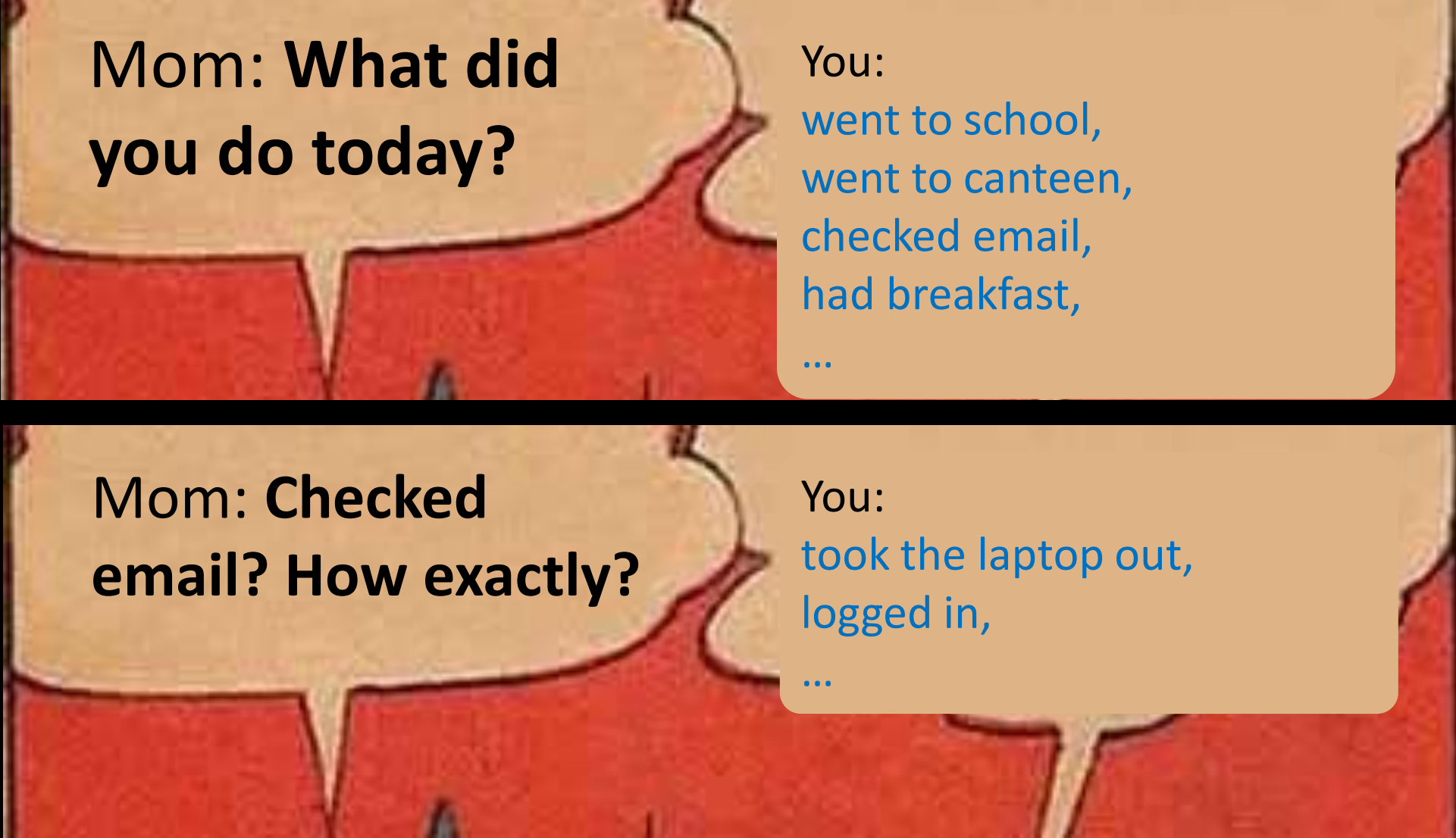
Mom: **What did
you do today?**

You:

walked to the bus stop,
got on the bus,
tapped the bus card,
walked to a seat,
sat down,

...





**Mom: What did
you do today?**

You:
went to school,
went to canteen,
checked email,
had breakfast,
...

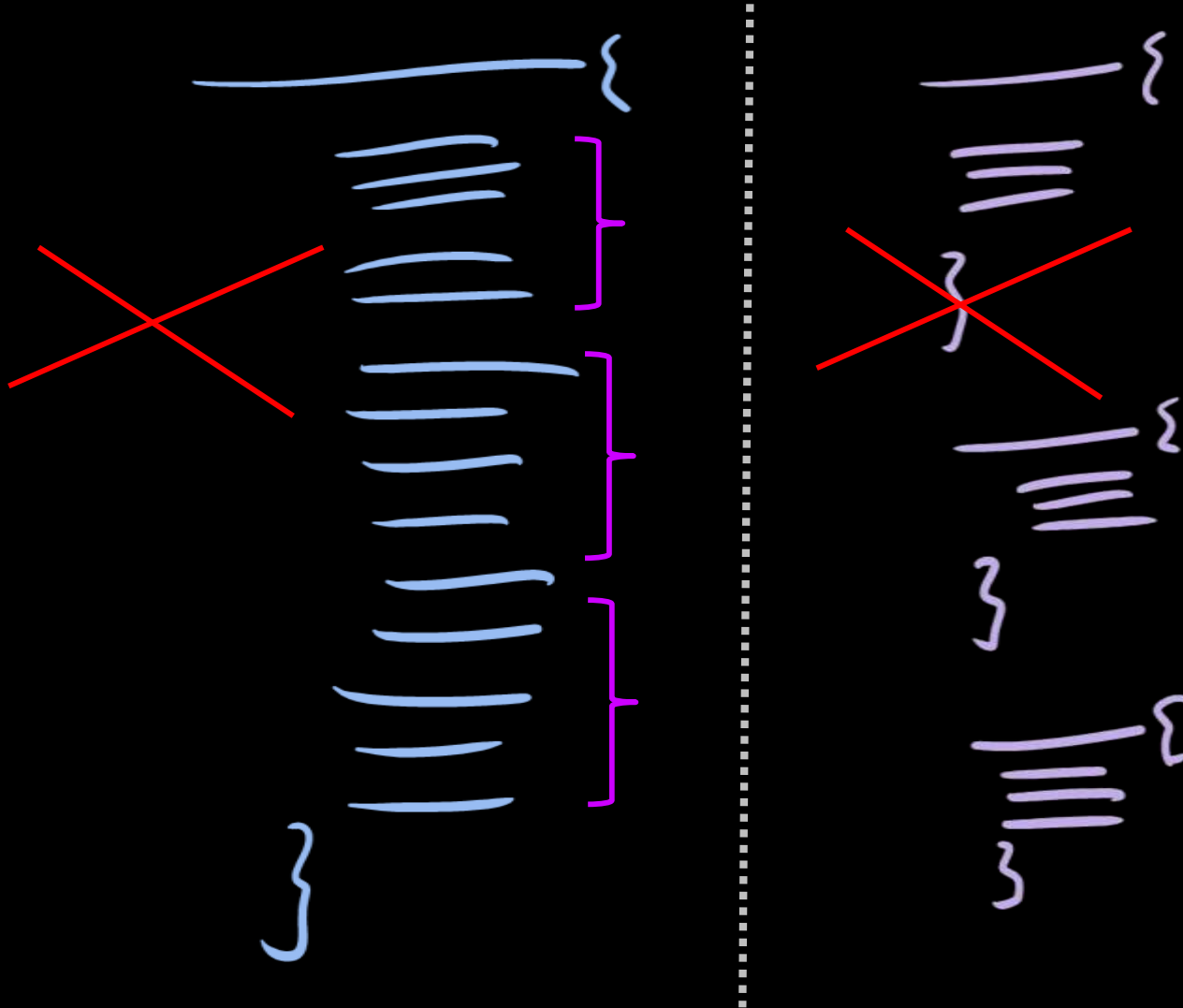
**Mom: Checked
email? How exactly?**

You:
took the laptop out,
logged in,
...

Single Level of
Abstraction Principle

SLAP ?

SLAP ?



Haphazard breaking into
smaller methods
is not SLAP!

Instead, extract the right
vocabulary to explain
the high-level logic

Q

How's SLAP here? How many levels? 0 1 2 3 4 5+

```
private static ArgumentMultimap extractArguments(String argsString, List<PrefixPosition> prefixPositions) {  
  
    // Sort by start position  
    prefixPositions.sort((prefix1, prefix2) -> prefix1.getStartPosition() - prefix2.getStartPosition());  
  
    // Insert a PrefixPosition to represent the preamble  
    PrefixPosition preambleMarker = new PrefixPosition(new Prefix(""), 0);  
    prefixPositions.add(0, preambleMarker);  
  
    // Add a dummy PrefixPosition to represent the end of the string  
    PrefixPosition endPositionMarker = new PrefixPosition(new Prefix(""), argsString.length());  
    prefixPositions.add(endPositionMarker);  
  
    // Map prefixes to their argument values (if any)  
    ArgumentMultimap argMultimap = new ArgumentMultimap();  
    for (int i = 0; i < prefixPositions.size() - 1; i++) {  
  
        // Extract and store prefixes and their arguments  
        Prefix argPrefix = prefixPositions.get(i).getPrefix();  
        String argValue = extractArgumentValue(argsString, prefixPositions.get(i), prefixPositions.get(i + 1));  
        argMultimap.put(argPrefix, argValue);  
    }  
  
    return argMultimap;  
}
```

So, is SLAP good enough in this case?

Does the `println` bother you?

What's going on here...? I can't understand this code because I don't see the code of this method 😞

```
private String showUser(String s) {  
    System.out.println(s);  
}
```

What if the code of the method was given here instead?
Easier to understand?

I don't want to split code into many methods because it makes the code hard to understand!

Q

Intention

```
private String showToUser(String s) {  
    System.out.println(s);  
}
```

Mechanism

Is this worth abstracting?

☐ yes

☐ no

☐ perhaps

Where to find examples of good quality code?

Not much easily accessible good quality code.

OSS code:

Readability may not be a priority.

Too big for beginners?



100644 | 203 lines (174 sloc) | 5.047 kb

```
package junit.textui;
```

```
import java.io.PrintStream;
```

```
import junit.framework.Test;
```

```
import junit.framework.TestCase;
```

```
import junit.framework.TestResult;
```

```
import junit.framework.TestSuite;
```

```
import junit.runner.BaseTestRunner;
```

```
import junit.runner.Version;
```

Javadoc comments

```
/**
```

```
 * A command line based tool to run tests.
```

```
 * <pre>
```

```
 * java junit.textui.TestRunner [-wait] TestCaseClass
```

```
 * </pre>
```

```
 *
```

```
 * <p>TestRunner expects the name of a TestCase class as argument.
```

```
 * If this class defines a static <code>suite</code> method it
```

```
 * will be invoked and the returned test is run. Otherwise all
```

```
 * the methods starting with "test" having no arguments are run.</p>
```

```
 *
```

```
 * <p>When the wait command line argument is given TestRunner
```

```
 * waits until the users types RETURN.</p>
```

```
 *
```

```
 * <p>TestRunner prints a trace as the tests are executed followed by a
```

```
 * summary at the end.</p>
```

```
 */
```

```
public class TestRunner extends BaseTestRunner {
```

```
    private ResultPrinter fPrinter;
```

```
    public static final int SUCCESS_EXIT= 0;
```

```
    public static final int FAILURE_EXIT= 1;
```

JUnit

```

87     @Override
88     public void testFailed(int status, Test test, Throwable t) {
89     }
90
91     @Override
92     public void testStarted(String testName) {
93     }
94
95     @Override
96     public void testEnded(String testName) {
97     }
98
99
100    /**
101     * Creates the TestResult to be used for the test run.
102     */
103    protected TestResult createTestResult() {
104        return new TestResult();
105    }
106
107    public TestResult doRun(Test test) {
108        // ... (code obscured by redaction) ...
109    }
110
111    public TestResult doRun(Test suite, boolean wait) {
112        TestResult result= createTestResult();
113        result.addListener(fPrinter);
114        long startTime= System.currentTimeMillis();
115        suite.run(result);
116        long endTime= System.currentTimeMillis();
117        long runTime= endTime-startTime;
118        fPrinter.print(result, runTime);
119
120
121        pause(wait);
122        return result;
123    }
124
125    protected void pause(boolean wait) {
126        if (!wait) return;
127        fPrinter.printWaitPrompt();

```

Indentation?

```

46     }
47
48     /**
49      * Constructs a TestRunner using the given ResultPrinter all the output
50      */
51
52     public TestRunner(ResultPrinter printer) {
53         fPrinter= printer;
54     }
55
56     /**
57      * Runs a suite extracted from a TestCase subclass.
58      */
59     static public void run(Class<? extends TestCase> testClass) {
60         run(new TestSuite(testClass));
61     }
62
63     /**
64      * Runs a single test and collects its results.
65      * This method can be used to start a test run
66      * from your program.
67      * <pre>
68      * public static void main (String[] args) {
69      *     test.textui.TestRunner.run(suite());
70      * }
71      * </pre>
72      */
73     static public TestResult run(Test test) {
74         TestRunner runner= new TestRunner();
75         return runner.doRun(test);
76     }
77
78     /**
79      * Runs a single test and waits until the user
80      * types RETURN.
81      */
82     static public void runAndWait(Test suite) {
83         TestRunner aTestRunner= new TestRunner();
84         aTestRunner.doRun(suite, true);
85     }
86

```

Very small
methods

```

32 public static final int EXCEPTION_EXIT= 2;
33
34 /**
35  * Constructs a TestRunner.
36  */
37 public TestRunner() {
38     this(System.out);
25 * summary at the end.</p>
26 */
27 public class TestRunner extends BaseTestRunner {
28     private ResultPrinter fPrinter;
29
30     public static final int SUCCESS_EXIT= 0;
31     public static final int FAILURE_EXIT= 1;
32     public static final int EXCEPTION_EXIT= 2;
33
34     /**
35      * Constructs a TestRunner.
36      */
37     public TestRunner() {
38         this(System.out);
39     }
40
41     /**
42      * Constructs a TestRunner using the given stream for all the output
43      */
44     public TestRunner(PrintStream writer) {
45         this(new ResultPrinter(writer));
46     }
47
48     /**
49     public TestRunner() {
50         this(System.out);
51     }
52
53     /**
54      * Constructs a TestRunner using the given stream for all the output
55      */
56     public TestRunner(PrintStream writer) {
57         this(new ResultPrinter(writer));

```

```

149  /**
150   * Starts a test run. Analyzes the command line arguments and runs the given
151   * test suite.
152   */
153  public TestResult start(String args[]) throws Exception {
154      String testCase= "";
155      String method= "";
156      boolean wait= false;
157
158      for (int i= 0; i < args.length; i++) {
159          if (args[i].equals("-wait"))
160              wait= true;
161          else if (args[i].equals("-c"))
162              testCase= extractClassName(args[++i]);
163          else if (args[i].equals("-m")) {
164              String arg= args[++i];
165              int lastIndex= arg.lastIndexOf('.');
166              testCase= arg.substring(0, lastIndex);
167              method= arg.substring(lastIndex + 1);
168          } else if (args[i].equals("-v"))
169              System.err.println("JUnit " + Version.id() + " by Kent Beck and Erich Gamma");
170          else
171              testCase= args[i];
172      }
173
174      if (testCase.equals(""))
175          throw new Exception("Usage: TestRunner [-wait] testCaseName, where name is the name of the TestCase class");
176
177      try {
178          if (!method.equals(""))
179              return runSingleMethod(testCase, method, wait);
180          Test suite= getTest(testCase);
181          return doRun(suite, wait);
182      } catch (Exception e) {
183          throw new Exception("Could not create and run test suite: " + e);
184      }
185  }
186
187  protected TestResult runSingleMethod(String testCase, String method, boolean wait) throws Exception {

```

Guard clause!


```

149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189

```

```

/**
 * Starts a test run
 * test suite.
 */
public TestResult sta
    String testCa
    String method
    boolean wait=

    for (int i= 0
        if (a

    else

    else i

    } else

    else

    }

    if (testCase.eq
        throw n

    try {
        if (!me

        Test su
        return c

    } catch (Excepti
        throw ne

    }

}

protected TestResult run

```

```

47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86

```

```

/**
 * Constructs a TestRunner using the given ResultPrinter all the output
 */
public TestRunner(ResultPrinter printer) {
    fPrinter= printer;
}

/**
 * Runs a suite extracted from a TestCase subclass.
 */
static public void run(Class<? extends TestCase> testClass) {
    run(new TestSuite(testClass));
}

/**
 * Runs a single test and collects its results.
 * This method can be used to start a test run
 * from your program.
 * <pre>
 * public static void main (String[] args) {
 *     test.textui.TestRunner.run(suite());
 * }
 * </pre>
 */
static public TestResult run(Test test) {
    TestRunner runner= new TestRunner();
    return runner.doRun(test);
}

/**
 * Runs a single test and waits until the user
 * types RETURN.
 */
static public void runAndWait(Test
    TestRunner aT

```

Good code
looks neat.

```
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86

/**
 * Constructs a TestRunner using the given ResultPrinter all the output
 */
public TestRunner(ResultPrinter printer) {
    fPrinter= printer;
}

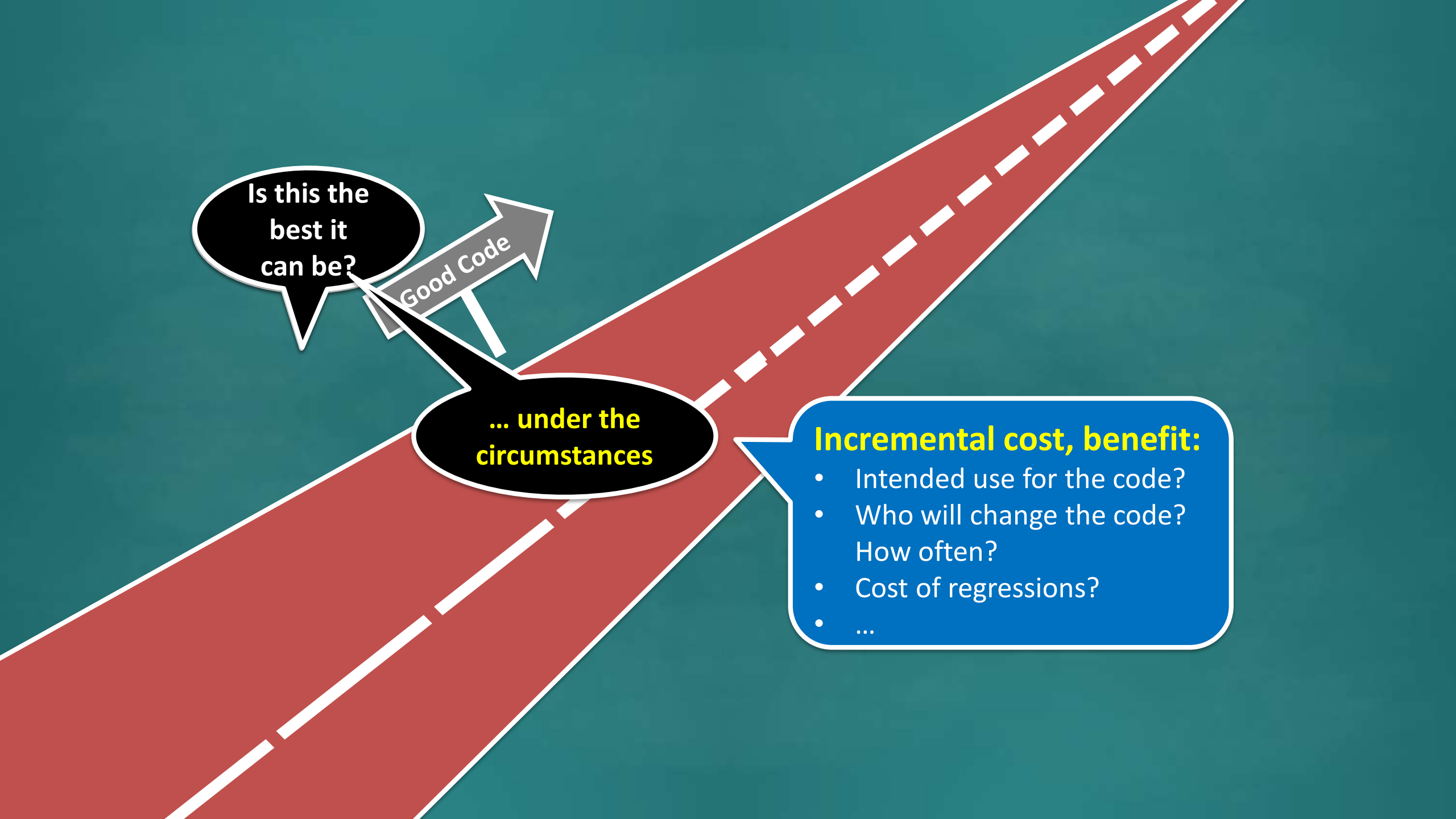
/**
 * Runs a suite extracted from a TestCase subclass.
 */
static public void run(Class<? extends TestCase> testClass) {
    run(new TestSuite(testClass));
}

/**
 * Runs a single test and collects its results.
 * This method can be used to start a test run
 * from your program.
 * <pre>
 * public static void main (String[] args) {
 *     test.textui.TestRunner.run(suite());
 * }
 * </pre>
 */
static public TestResult run(Test test) {
    TestRunner runner= new TestRunner();
    return runner.doRun(test);
}

/**
 * Runs a single test and waits until the user
 * types RETURN.
 */
static public void runAndWait(Test
    TestRunner aTest
    aTest
```







Is this the
best it
can be?

Good Code

**... under the
circumstances**

Incremental cost, benefit:

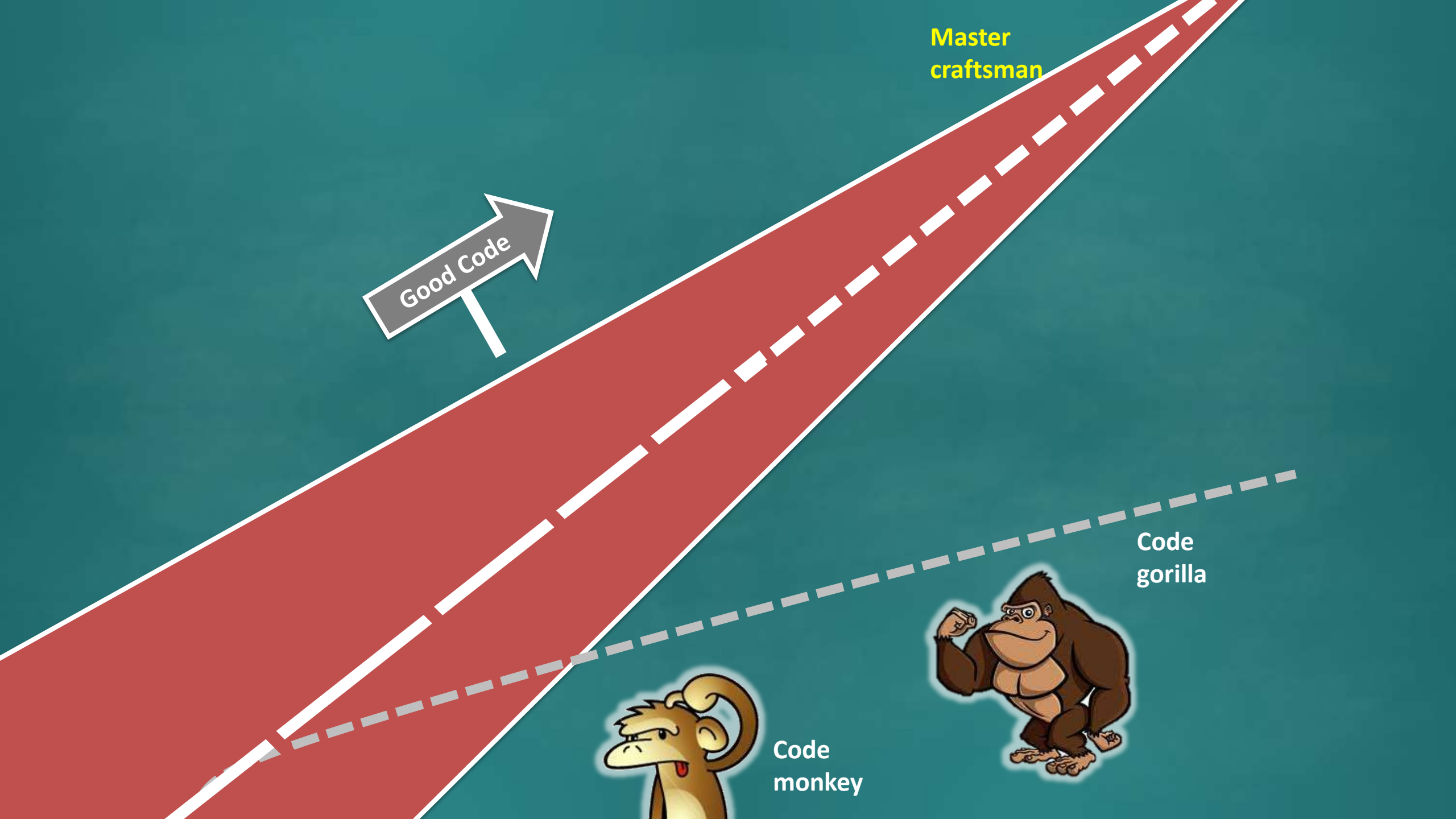
- Intended use for the code?
- Who will change the code?
How often?
- Cost of regressions?
- ...



Code
monkey



Code
gorilla



Master
craftsman

Good Code

Code
gorilla



Code
monkey



Q

The aim of this module is,

a

Master craftsman



b

Code monkey



c

Code gorilla



Q

What's the best paradigm?

- ☐ Procedural
- ☐ OOP
- ☐ Functional
- ☐ A mix

It depends:

e.g., on the application domain, tech stack , the nature of data sources, expertise of future devs, ...

Good OOP != OOP Good

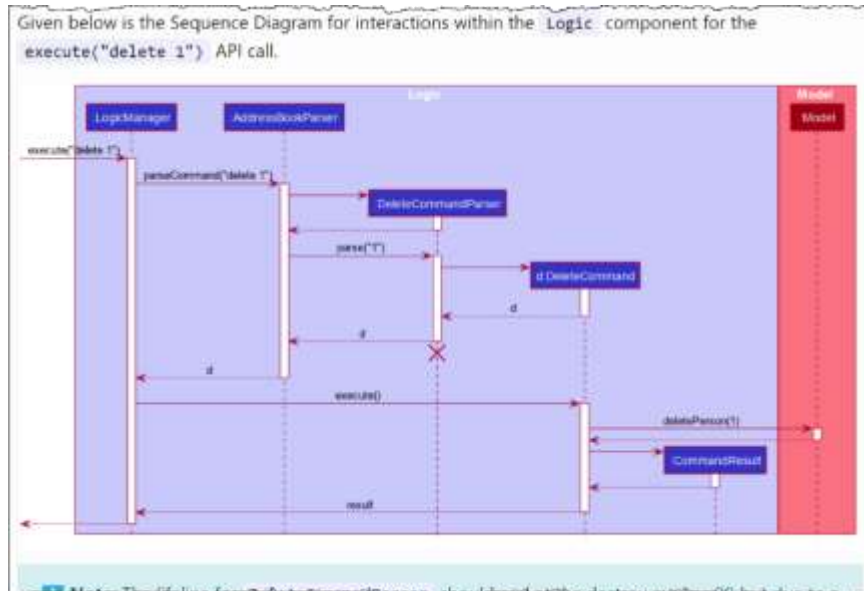
learn OOP well, push it to its limits, learn pros/cons

Week 6

Interpreting
only!

Topics:

- [W6.1] Modeling: Sequence Diagrams ⚡⚡
- [W6.2] Architecture Diagrams ⚡
- [W6.3] IDEs: Intermediate Features ⚡⚡
- [W6.4] Logging ⚡⚡



Admin:

1. [optional] Submit midterm feedback for the course ⌚ Sat, Feb 10, 10:00 AM
2. Submit weekly quiz P

iP:

1. Add Increments: A-BetterGui, A-Personality, A-MoreErrorHandling, A-MoreTesting, A-AiAssisted
2. Finalize the features
3. Set up a product website
4. Submit the final version ⌚ Fri, Feb 10, 10:00 AM

tP:

1. 👤 Conceptualize the MVP version
2. 👤 Draft the feature specification ⌚ Sun, Feb 12, 10:00 AM
3. 👤 Set up the project repo
4. 👤 Get familiar with the codebase

Week 6

Topics:

- [W6.1] Modeling: Sequence Diagrams ⚡⚡
- [W6.2] Architecture Diagrams ⚡
- [W6.3] IDEs: Intermediate Features ⚡⚡
- [W6.4] Logging ⚡⚡

Admin:

1. [optional] Submit midterm feedback for the course ⌚ Sat, Feb 14, 10:00 AM
2. Submit weekly quiz P

iP:

1. Add Increments: A-BetterGui, A-Personality, A-MoreErrorHandling, A-MoreTesting, A-AiAssisted
2. Finalize the features
3. Set up a product website
4. Submit the final version ⌚ Fri, Feb 13, 10:00 PM

tP:

1. 👤 Conceptualize the MVP version
2. 👤 Draft the feature specification ⌚ Sun, Feb 15, 10:00 PM
3. 👤 Set up the project repo
4. 👤 Get familiar with the codebase

It's your primary
weapon.

Soon:
Learn basics first



It's your primary
weapon.

Soon:
Learn basics first

Long term:
**Become an
IDE ninja**



Q

You **must** use IntelliJ IDEA in this module

☐ True

☐ False

Can use alternatives;
productivity is the actual goal

But must be aware of and know
how to use IDEs

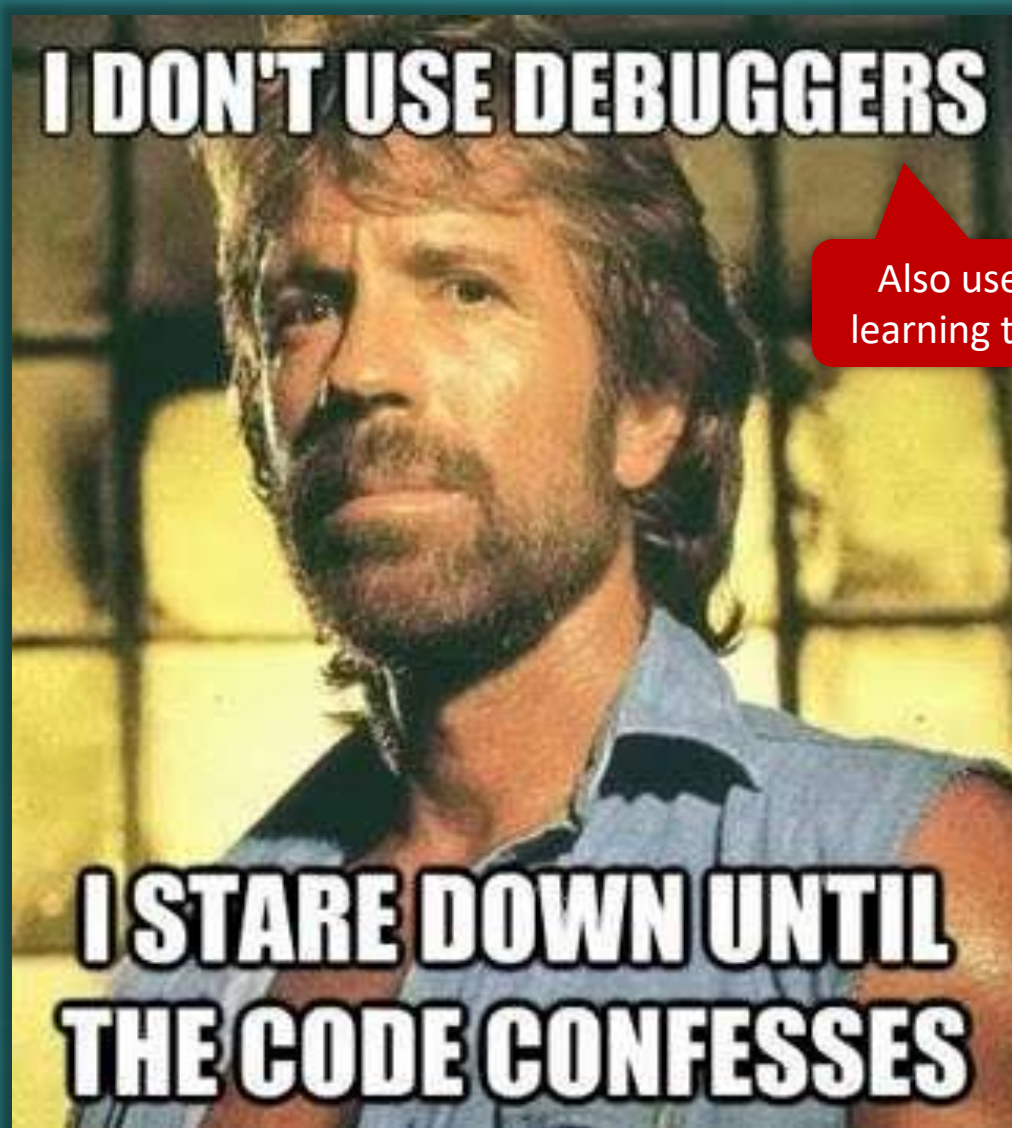
It's your primary
weapon.

Soon:
Learn basics first

Semester-end:
**Become an
IDE ninja**



'eyeballing' code to find the bug



Also useful for
learning the code

```
701
702  /**
703   * Creates storage file if it does not exist. Shows feedback to user.
704   *
705   * @param filePath file to create if not present
706   */
707  private static void createFileIfMissing(String filePath) {  filePath: "addressbook.txt"
    final File storageFile = new File(filePath);  storageFile: "addressbook.txt" file
    if (storageFile.exists()) {  storageFile: "addressbook.txt"
        return;
    }

    showToUser(String.format(MESSAGE_ERROR_MISSING_STORAGE_FILE, filePath));

    try {
        storageFile.createNewFile();
        showToUser(String.format(MESSAGE_STORAGE_FILE_CREATED, filePath));
    } catch (IOException ioe) {
        showToUser(String.format(MESSAGE_ERROR_CREATING_STORAGE_FILE, filePath));
        exitProgram();
    }
}

712
713
714
715
716
717
718
719
720
721
722
723
```

AddressBook > createFileIfMissing()

Debug AddressBook

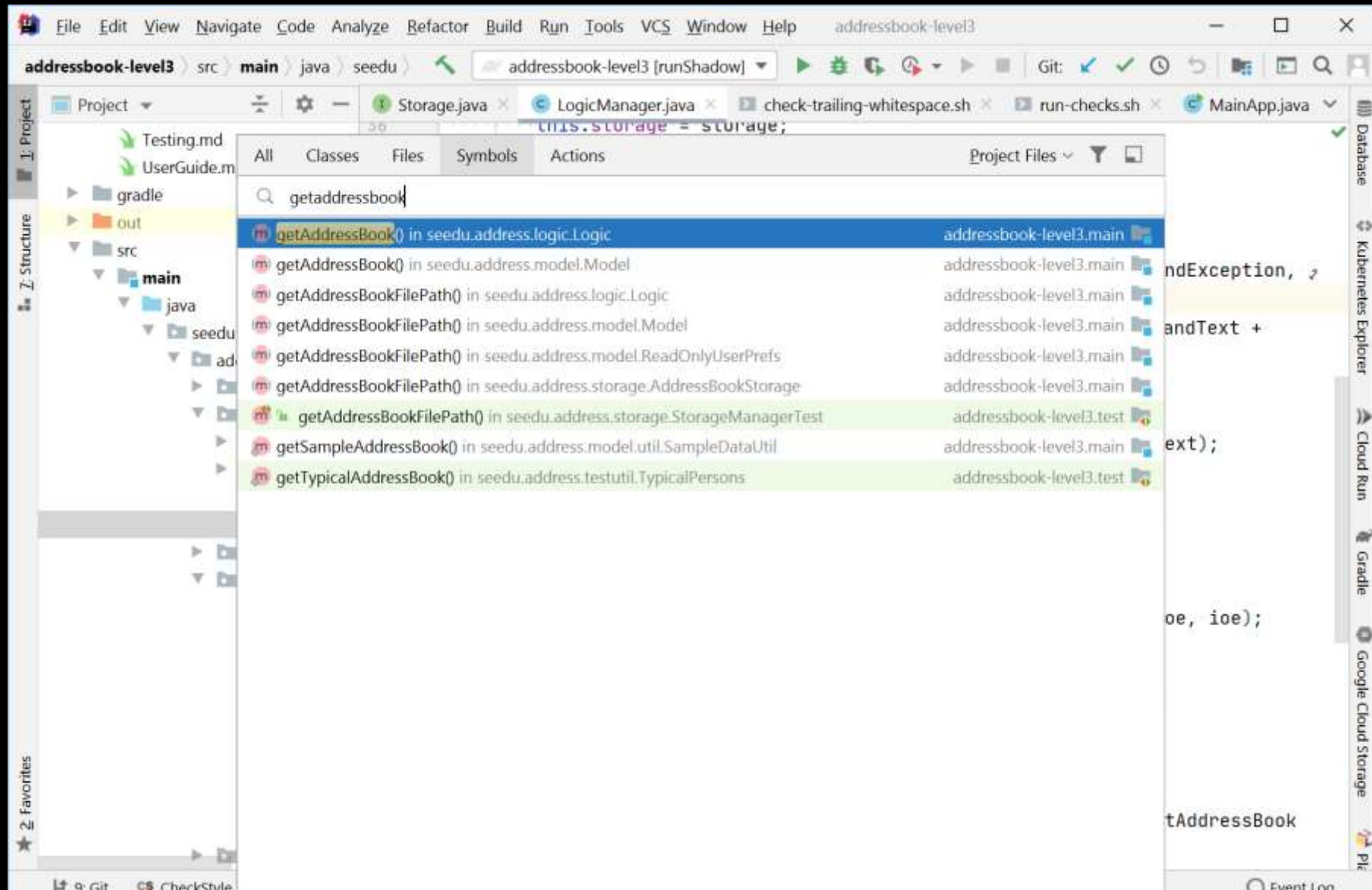
Debugger Console →

Frames →

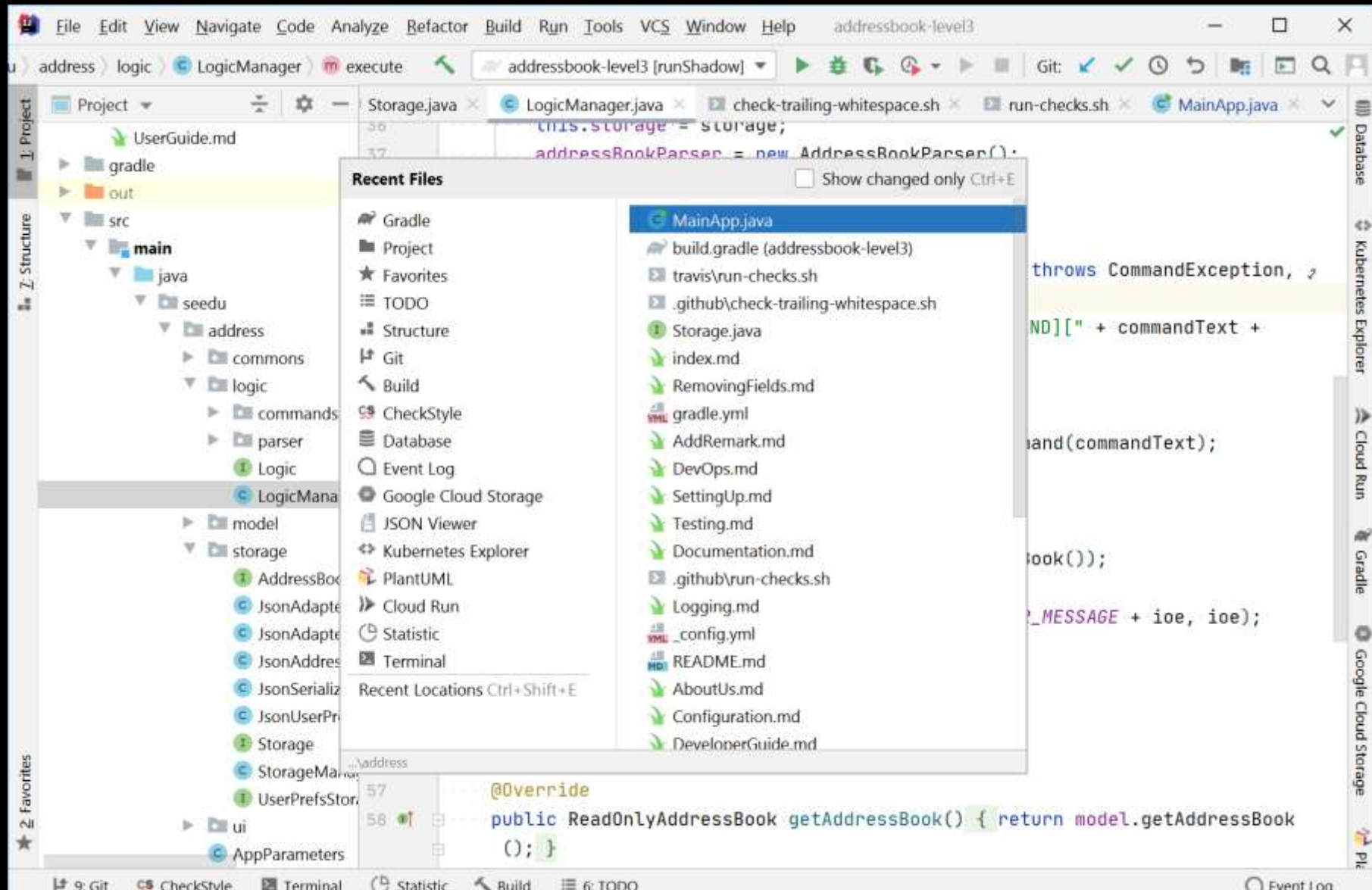
- "main"@1 in group "main": RUNNING
- createFileIfMissing:709, AddressBook (seedu.addressbook)
- setupDefaultFileForStorage:306, AddressBook (seedu.addressbook)
- processProgramArgs:270, AddressBook (seedu.addressbook)
- main:211, AddressBook (seedu.addressbook)

Variables

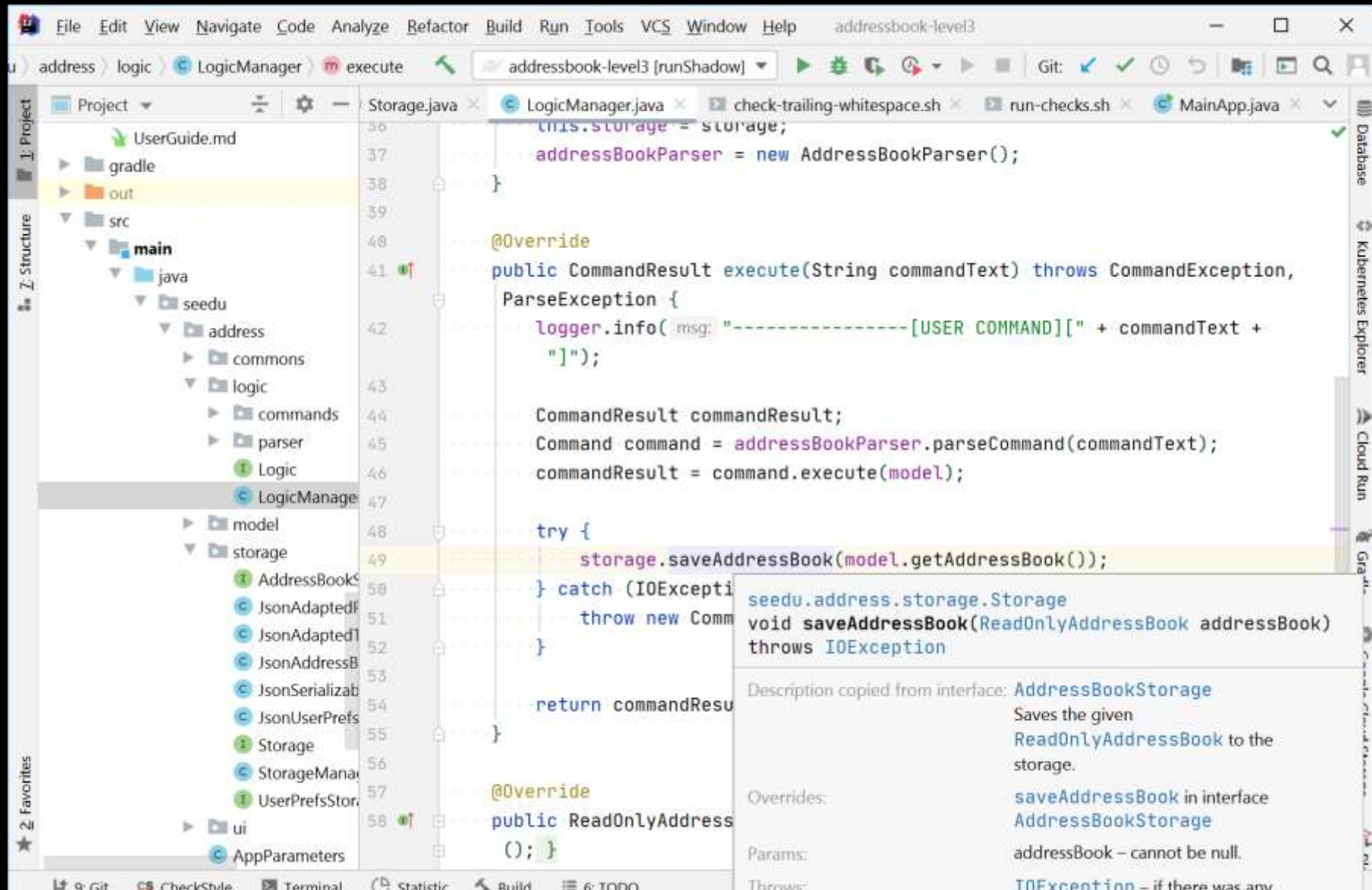
- static members of AddressBook
- filePath = "addressbook.txt"
- storageFile = {File@642} "addressbook.txt"



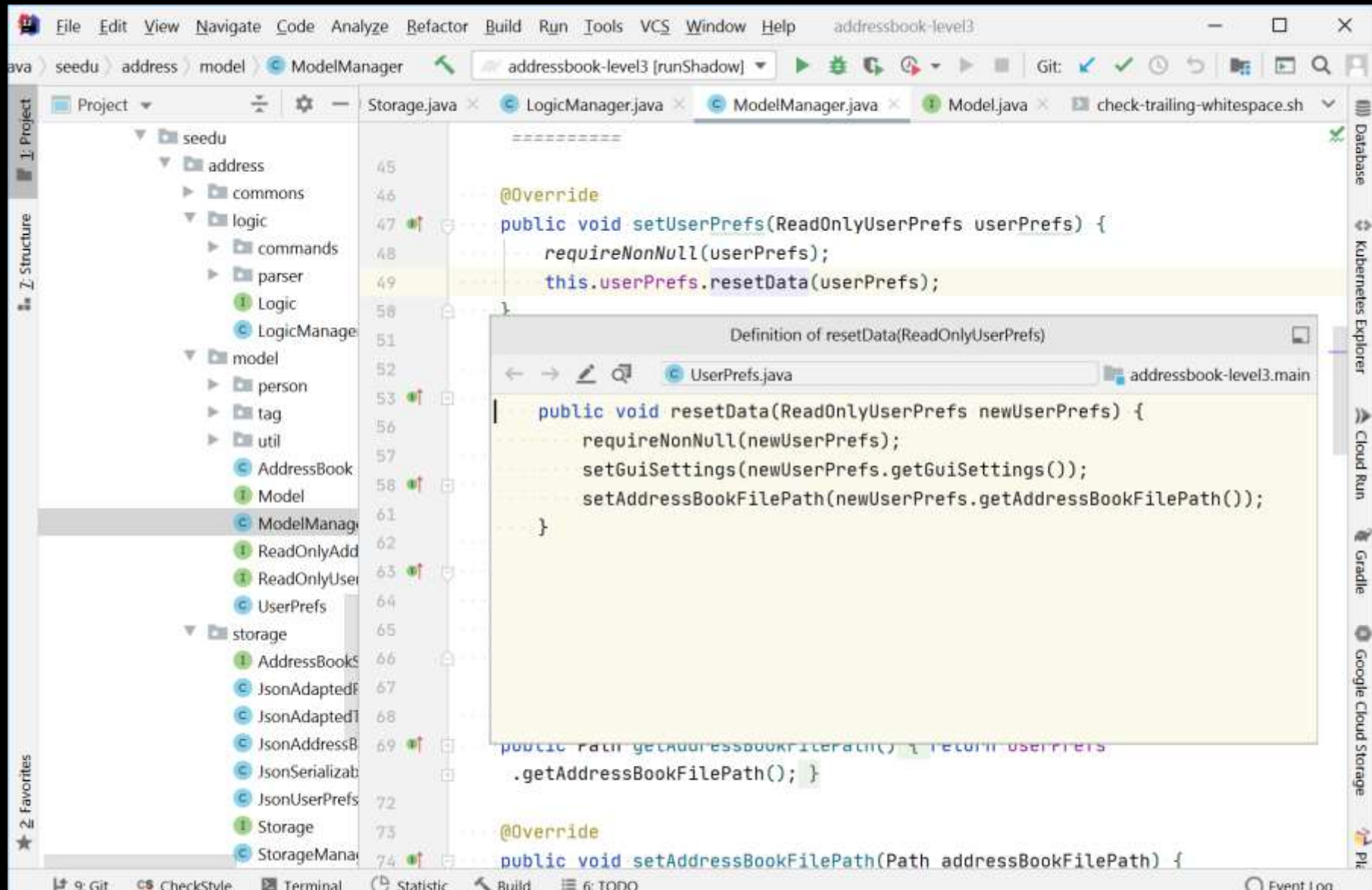
Search for symbol (Windows): Ctrl + Alt + Shift + N



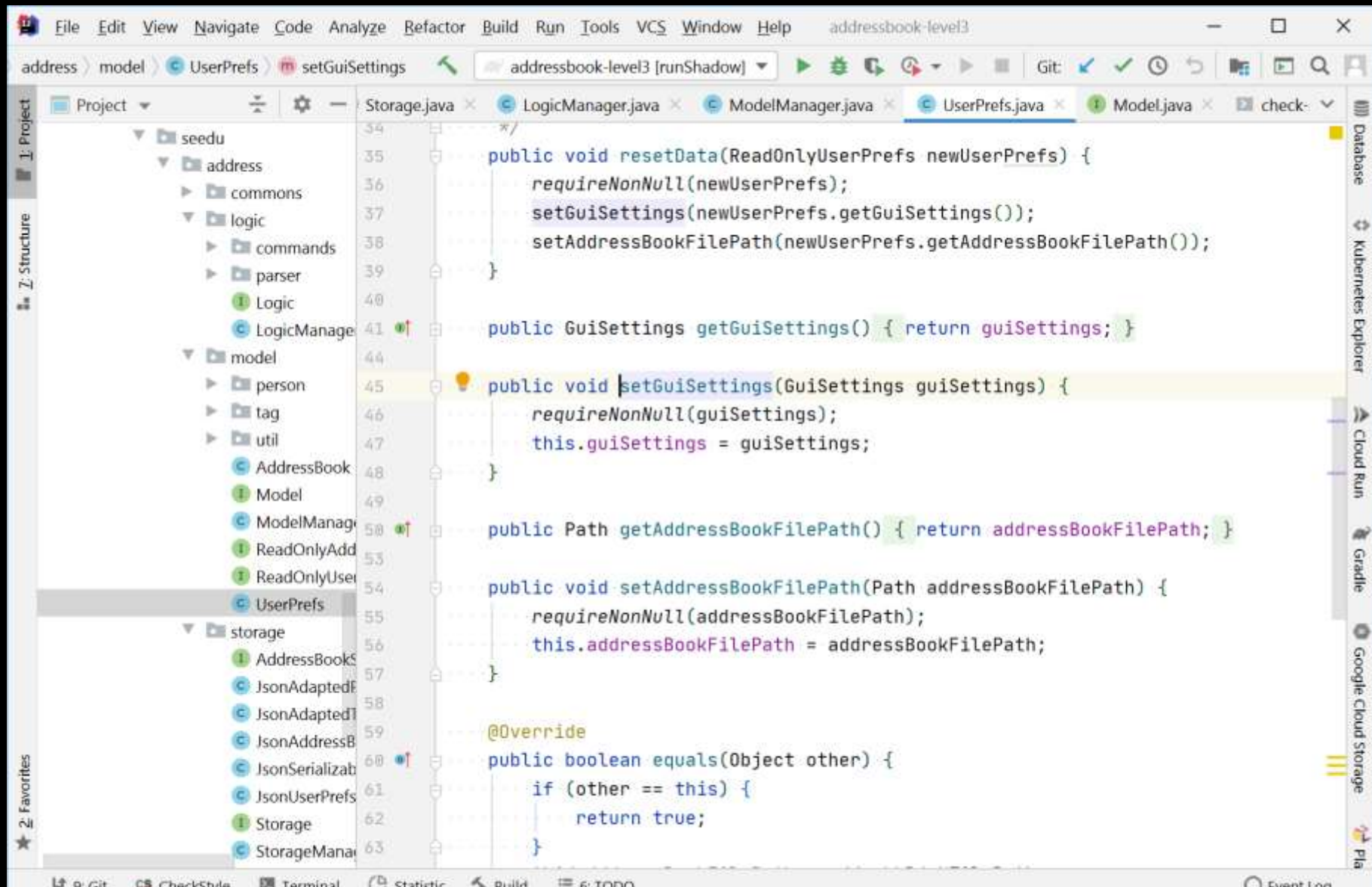
Show recent files (Windows): Ctrl + E



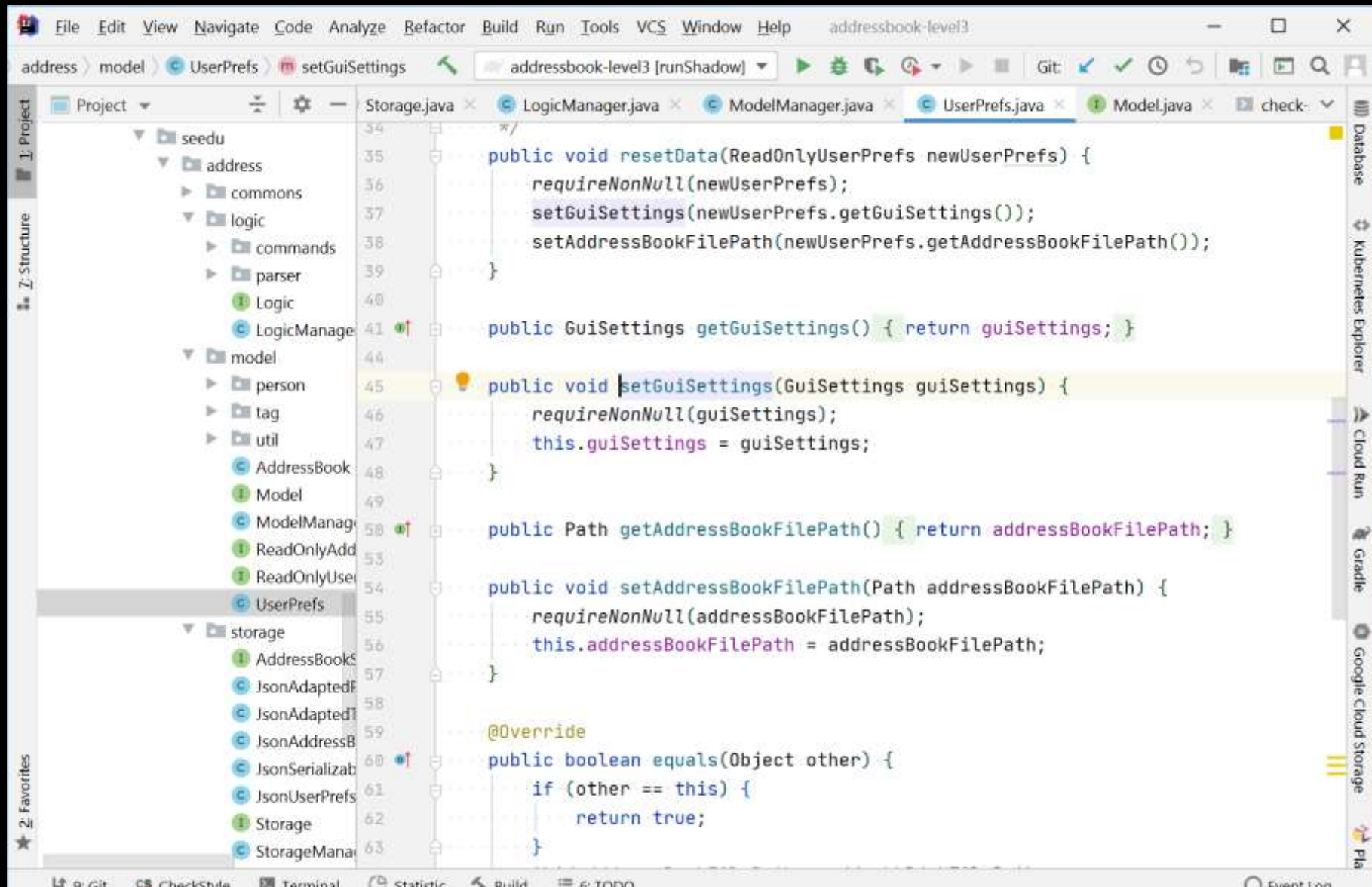
Quick documentation (Windows): Ctrl + Q



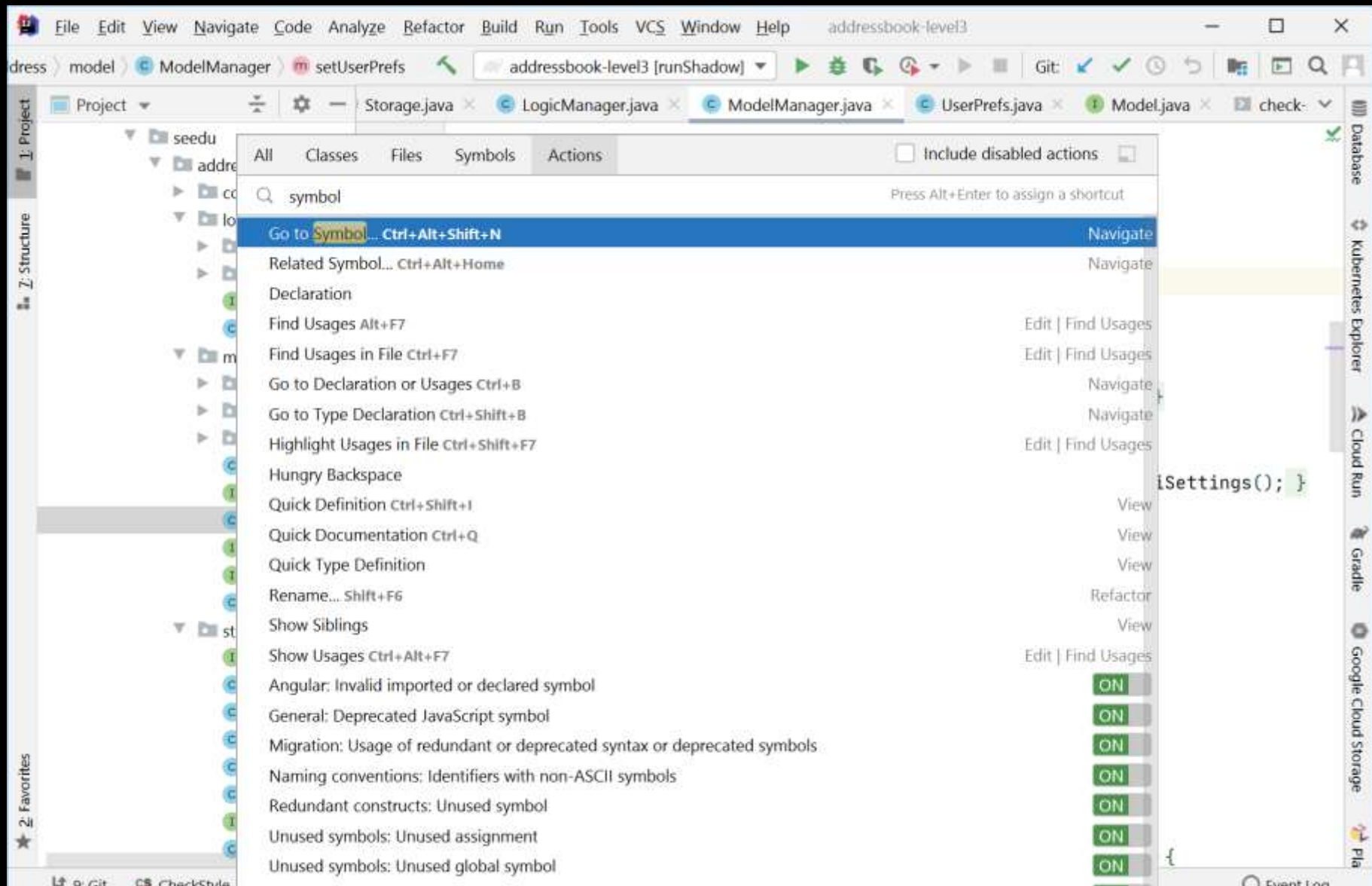
Show definition (Windows): Ctrl + Shift + I



Go to definition (Windows): Ctrl + B



Go back (Windows): Ctrl + Alt + ← / →



Search actions (Windows): Ctrl + Shift + A

Some code navigation features

Find Class : **Ctrl + N** (supports CamelCase too)

Find File: **Ctrl + Shift + N** (type \ to find folders)

Find Symbol : **Ctrl + Alt + Shift + N**

Quick Documentation: **Ctrl+Q**

Quick Implementation: **Ctrl + Shift + I**

Navigate to Declaration: **Ctrl + B**

Go back/forward: **Ctrl + Alt + →/←**

How to remember shortcuts:

- * Search shortcuts: **Ctrl + Shift + A**

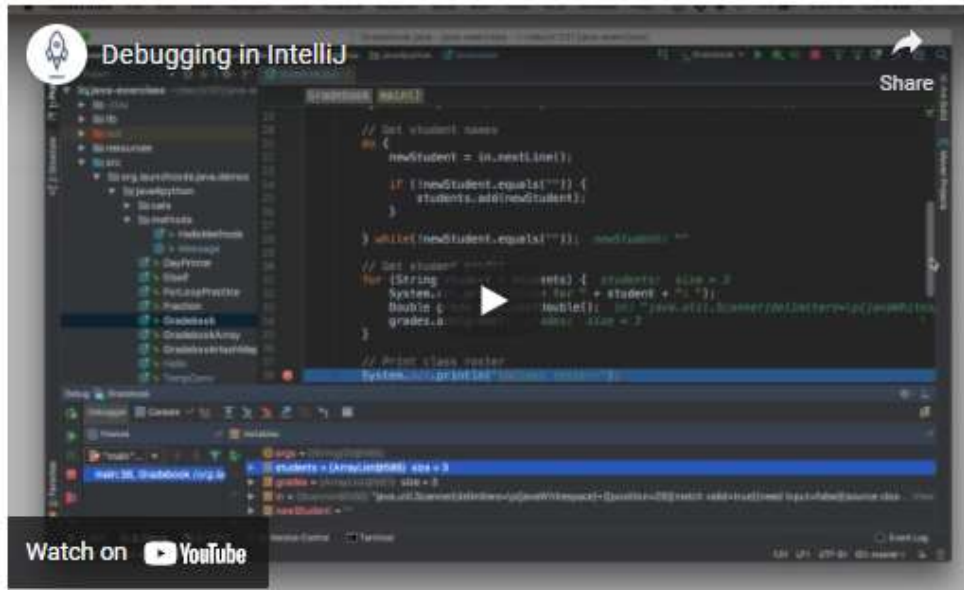
- * Say the shortcut (in your mind) every time you use it.

W6.3b ★★★

Tools → IntelliJ IDEA → Debugging: Basic

🔧 Can step through a program using a debugger

This video (from LaunchCode) gives a pretty good explanation of how to use the IntelliJ IDEA debugger.



- [IntelliJ IDEA Documentation: Debugging Basics](#) - Can be used as a reference document when you want to recall how to use a debugging feature.

W6.3c ★★

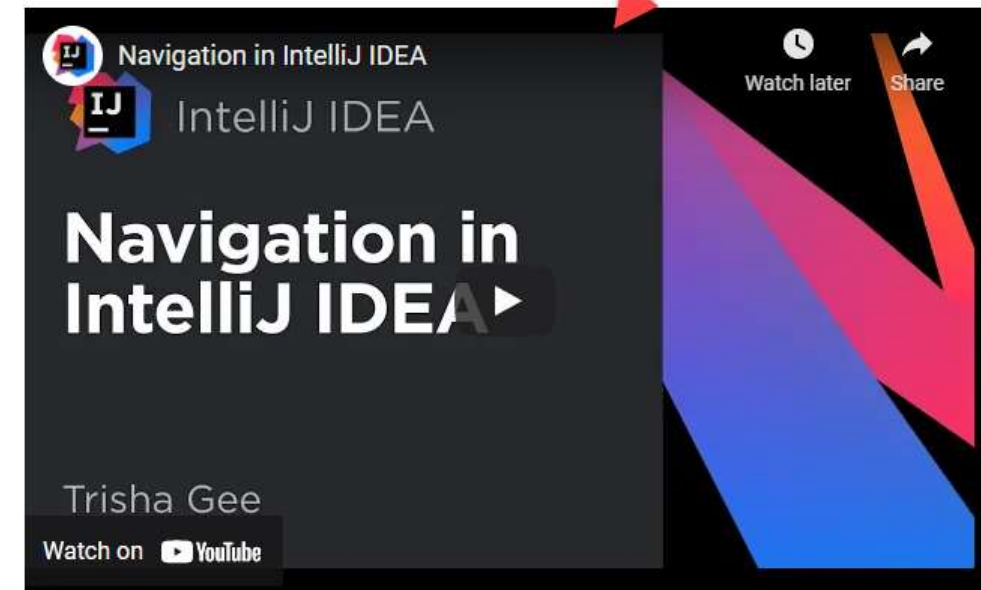
Tools → IntelliJ IDEA → Code navigation

🔧 Can navigate code effectively using IDE features

Some useful navigation shortcuts:

1. Quickly locate a file by name.
2. Go to the definition of a method from where it is used.
3. Go back to the previous location.
4. View the documentation of a method from where the method is being used, without navigating to the method itself.
5. Find where a method/field is being used.

IntelliJ IDEA Code Navigation



Week 6

Topics:

- [W6.1] Modeling: Sequence Diagrams ⚡⚡
- [W6.2] Architecture Diagrams ⚡
- [W6.3] IDEs: Intermediate Features ⚡⚡
- [W6.4] Logging ⚡⚡

Admin:

1. [optional] Submit midterm feedback for the course ⌚ Sat, Feb 14, 10:00 AM
2. Submit weekly quiz P

iP:

1. Add Increments: A-BetterGui, A-Personality, A-MoreErrorHandling, A-MoreTesting, A-AiAssisted
2. Finalize the features
3. Set up a product website
4. Submit the final version ⌚ Fri, Feb 13, 10:00 PM

tP:

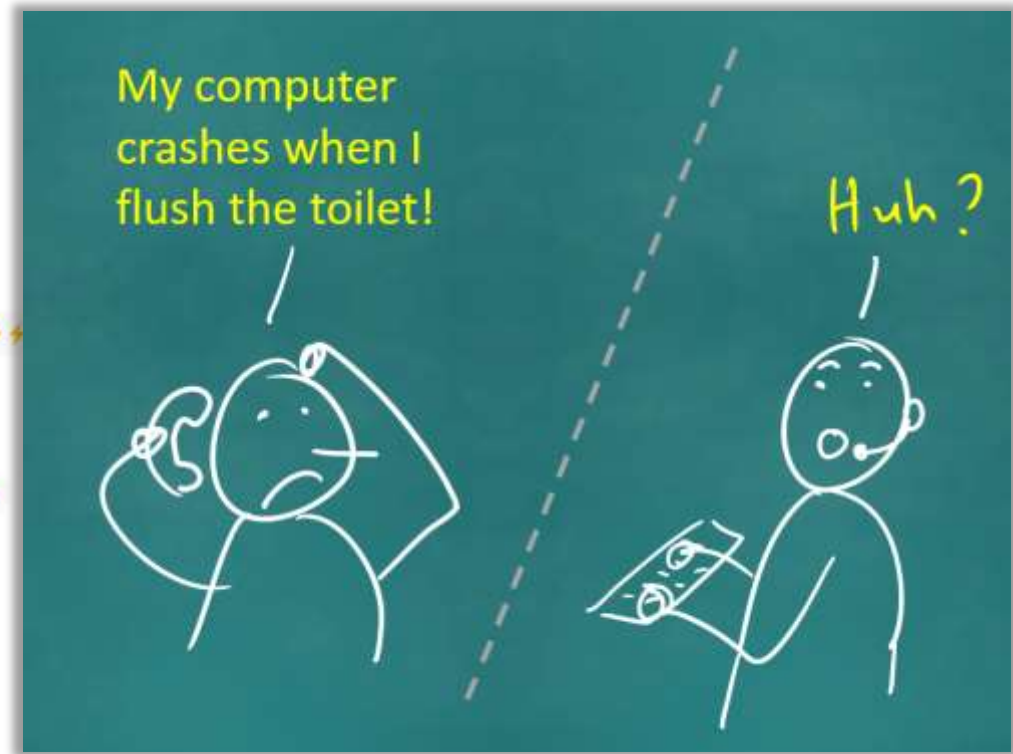
1. 👤 Conceptualize the MVP version
2. 👤 Draft the feature specification ⌚ Sun, Feb 15, 10:00 PM
3. 👤 Set up the project repo
4. 👤 Get familiar with the codebase

There are some concrete deliverables for this item

Week 6

Topics:

- [W6.1] Modeling: Sequence Diagrams ⚡⚡
- [W6.2] Architecture Diagrams ⚡
- [W6.3] IDEs: Intermediate Features ⚡⚡
- [W6.4] Logging ⚡⚡



3. Set up a product website

4. Submit the final version ⌚ Fri, Feb 14, 2020

tP:

1. 👤 Conceptualize the MVP version

2. 👤 Draft the feature specification

⌚ Sun, Feb 16, 2020

3. 👤 Set up the project repo

4. 👤 Get familiar with the codebase

Week 6

Topics:

- [W6.1] Modeling: Sequence Diagrams ⚡⚡
- [W6.2] Architecture Diagrams ⚡
- [W6.3] IDEs: Intermediate Features ⚡⚡
- [W6.4] Logging ⚡⚡

Admin:

1. [optional] Submit midterm feedback for the course ⌚ Sat, Feb 10, 10:00 AM - 11:00 AM
2. Submit weekly quiz P

iP:

1. Add Increments: A-BetterGui, A-Personality, A-MoreErrorHandling, A-MoreTesting, A-AiAssisted
2. Finalize the features
3. Set up a product website
4. Submit the final version ⌚ Fri, Feb 9, 10:00 AM - 11:00 AM

tP:

1. 👥 Conceptualize the MVP version
2. 👥 Draft the feature specification ⌚ Sun, Feb 11, 10:00 AM - 11:00 AM
3. 👥 Set up the project repo
4. 👤 Get familiar with the codebase

Week 6

Topics:

- [W6.1] Modeling: Sequence Diagrams ⚡⚡
- [W6.2] Architecture Diagrams ⚡
- [W6.3] IDEs: Intermediate Features ⚡⚡
- [W6.4] Logging ⚡⚡

Admin:

1. [optional] Submit midterm feedback for the course ⌚ Sat, Feb 10, 2024 10:00 AM - 11:00 AM
2. Submit weekly quiz P

iP:

1. Add Increments: A-BetterGui, A-Personality, A-MoreErrorHandling, A-MoreTesting, A-AiAssisted
2. Finalize the features
3. Set up the infrastructure
4. Submit

Make it as small as possible, but can add more if time left

tP:

1. 👥 Conceptualize the MVP version
2. 👥 Draft the feature specification ⌚ Sun, Feb 11, 2024 10:00 AM - 11:00 AM
3. 👥 Set up the project repo
4. 👤 Get familiar with the codebase



*Thank
you!*

Dr Ganesh Neelakanta Iyer

gni@nus.edu.sg

<http://ganeshniyer.github.io>

<https://www.linkedin.com/in/ganeshniyer/>